so that we can modify the hostname.

```
func run() {
    // Stuff that we previously went
over

    cmd.SysProcAttr = &syscall.
SysProcAttr{
        Cloneflags: syscall.CLONE_NEWUTS,
    }

    must(cmd.Run())
}
```

Running it returns what is shown in Figure 4.

Awesome! As per Figure 4, we now can modify the hostname in our container-like environment without letting the host environment change.

But if we observe closely, our process IDs within the container are still the same. We're able to see the processes running in our host OS even from within our container.

To fix this, we need to use the PID namespace. As discussed above, the PID namespace will allow us process isolation.

```
func run() {
    // Stuff that we previously went
over

    cmd.SysProcAttr = &syscall.
SysProcAttr{
        Cloneflags: syscall.CLONE_NEWUTS
| syscall.CLONE_NEWPID,
    }

    must(cmd.Run())
}
```

However, unlike the case of UTS namespace, simply adding the PID namespace here like this won't help. We will have to create another copy of our process so that it can be run with PID 1:

```
func run() {
    cmd := exec.Command("/proc/self/
exe", append([]string{"child"},
os.Args[2:]...)...)
    cmd.Stdin = os.Stdin
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    cmd.SysProcAttr = &syscall.
SysProcAttr{
        Cloneflags: syscall.CLONE_NEWUTS
| syscall.CLONE_NEWPID,
    }

    must(cmd.Run())
}

func child() {
    fmt.Printf("Running %v as PID
%d\n", os.Args[2:], os.Getpid())

    cmd := exec.Command(os.Args[2],
os.Args[3:]...)
    cmd.Stdin = os.Stdin
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    must(cmd.Run())
}

func main() {
    switch os.Args[1] {
    case "run":
        run()
    case "child":
        child()
    default:
        panic("I'm sorry, what?")
    }
}
```

```
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# ps
    PID TTY          TIME CMD
   2930 pts/0    00:00:00 sudo
   2931 pts/0    00:00:00 bash
   3254 pts/0    00:00:00 ps
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# hostname
lubuntu
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# go run main.go run bash
Running [bash]
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# ps
    PID TTY          TIME CMD
   2930 pts/0    00:00:00 sudo
   2931 pts/0    00:00:00 bash
   3256 pts/0    00:00:00 go
   3289 pts/0    00:00:00 main
   3296 pts/0    00:00:00 bash
   3303 pts/0    00:00:00 ps
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# hostname
lubuntu
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# hostname pratikms
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# hostname
pratikms
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# exit
exit
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers# hostname
lubuntu
root@lubuntu:/home/lubuntu/Projects/make-sense-of-containers#
```

Figure 4: Modifying hostname in our container and confirming that host OS is not affected

What we are doing is that whenever we run *go run main.go run bash*, our *main()* function will be called. As the value of *os.Args[1]* will be 'run' at this instance, it will call our *run()* function. Within *run()*, we are using */proc/self/exe* to create a copy of our current process. We are essentially creating a copy and calling it again by appending the string 'child' to it, followed by the rest of the arguments that we received in *run()*. When we do this, our *main()* function will be invoked again, with the difference being that the value of *os.Args[1]* will be 'child' this time. From there on, the rest of the script executes as we saw before.

But, even after all this, do we really have process isolation? Do we have resource limiting? Answers to that and more will be covered in the next part of this series. Watch out for this space! END

By: Pratik Shivaraikar

The author is an open source software enthusiast, evangelist, and contributor with varied experience of working in the storage, security, and wireless domains. He actively works on Go, Python, Node.js, PHP, and more. He writes at *https://blog.pratikms.com*.